SuperAdmin Security Features & Security Testing Documentation

Health Database / Multi-Tenant Facility Management System — 7 September 2026

1. Purpose

This document records the security controls implemented for the SuperAdmin layer and the security/authorization attacks and functional tests performed during hardening.

2. SuperAdmin Security Features

#	Security Feature	Implementation / Security Purpose
1	Firebase Authentication	SuperAdmin access begins with Firebase Authentication; the backend verifies the Firebase ID token.
2	Server-Side SuperAdmin Verification	Protected requests verify Firebase UID/email against the Mother DB profile and require SuperAdmin clearance, initialized account status, and master facility context.
3	Flask SuperAdmin Session	Successful Firebase verification establishes a server-side Flask session; active facility context is maintained server-side.
4	CSRF Protection	State-changing SuperAdmin requests require a CSRF security token; missing or invalid tokens are rejected.
5	Server-Side Facility Isolation	Tenant-sensitive operations derive facility context from the authenticated server session rather than trusting a client-supplied facility ID.
6	Tenant Firebase Admin SDK	Facility user management uses the facility's server-side service-account configuration and validates the credential/project mapping.
7	Fail-Closed Tenant Initialization	Missing or mismatched tenant credentials do not fall back to project-ID-only access.
8	Protected Facility Management	Facility listing, creation, update, and deletion are SuperAdmin-protected; master-facility safeguards and dependency checks are applied.
9	Protected User Management	Provisioning, updating, password reset, and deletion are SuperAdmin-protected and constrained to the active tenant.
10	Privilege Escalation Prevention	Provisioning and update endpoints reject attempts to assign the SuperAdmin permission.
11	Safe API Responses	Protected APIs return controlled responses and do not expose service-account private keys.
12	Authenticated Logout	Logout clears the Flask session and signs out Firebase before returning to the SuperAdmin login page.

3. Security Attack & Validation Tests

#	Test / Attack	Method	Observed Result	Status
1	Unauthenticated facility API	curl request to /api/facilities with no Authorization header.	HTTP 401; Missing Firebase authorization token.	PASS
2	Fake Firebase token	Invalid/fabricated Firebase Bearer token submitted to a protected endpoint.	HTTP 401; token rejected.	PASS
3	Master facility spoof	Attempted to select master as an active tenant.	HTTP 400; valid tenant facility required.	PASS
4	Cross-facility DELETE spoof	Forged facility_id=master while active facility was lakan_cc.	Operation remained constrained to active lakan_cc.	PASS
5	Cross-facility UPDATE spoof	Forged facility ID during an existing-user update.	Server derived active facility; target outside it was not accepted.	PASS
6	Provisioning facility spoof	Submitted facility_id=master while active facility was lakan_cc.	Server used active lakan_cc; tenant isolation held.	PASS
7	Password-reset invalid UID	Submitted a nonexistent/test UID.	Selected account not found; no password change.	PASS
8	UID/email mismatch	Real disposable UID paired with a different email.	Target profile not found in active tenant.	PASS
9	Provisioning privilege escalation	Submitted permission=SuperAdmin.	Rejected; only Viewer, Editor, or Admin accepted.	PASS
10	Update privilege escalation	Submitted permission=SuperAdmin during update.	Rejected as invalid permission.	PASS
11	Missing CSRF token	Removed CSRF token from protected state-changing request.	Rejected with Missing CSRF security token.	PASS
12	Invalid CSRF token	Submitted invalid CSRF token to protected update endpoint.	Rejected with Invalid CSRF security token.	PASS

#	Test / Attack	Method	Observed Result	Status
13	Cross-facility password reset	Forged facility_id=master while active facility was lakan_cc.	Operation remained within active tenant.	PASS
14	Final unauthenticated endpoint test	Logged out, then directly requested /api/facilities with curl.	HTTP 401; no facility data disclosed.	PASS
15	Facility creation functional test	Created an additional facility through the SuperAdmin dashboard.	Facility creation completed successfully.	PASS

4. Test Results Summary

15 validation tests were documented: 14 security/authorization tests and 1 functional facility-creation test. All documented security tests passed based on observed responses and server-side behavior.

Category	Tests	Result
Authentication / session boundary	2	PASS
Tenant isolation / facility spoofing	5	PASS
User identity validation	2	PASS
Privilege escalation	2	PASS
CSRF protection	2	PASS
Facility creation	1	PASS
Total	15	PASS

5. Security Notes and Limitations

• Testing was performed against the local SuperAdmin application using disposable/test accounts and localhost endpoints.

• Tests focused on authentication, authorization boundaries, tenant isolation, CSRF enforcement, and privilege-escalation resistance.

• A PASS records observed behavior at test time; it is not a claim of complete security or a substitute for a formal penetration test.

• The final unauthenticated API test exercised the backend directly rather than relying on dashboard UI restrictions.

• Successful creation of an additional facility confirms that the protected facility-management workflow is operational.

6. Conclusion

The SuperAdmin layer demonstrated effective enforcement of authentication, server-side authorization, CSRF protection, tenant isolation, privileged-operation controls, and facility-management functionality during the documented local validation tests. The final direct unauthenticated API test returned HTTP 401 and disclosed no facility registry data.